

Proyecto Individual: Desarrollo de una aplicación para la generación de gráficos y texto

Marco Alonso Rodríguez Villegas

Ingeniería en Computadores

Carnet: 2020168039

markorovi24@estudiantec.cr

Abstract—El presente proyecto tiene como objetivo el desarrollo de una aplicación para la interpolación bilineal de imágenes utilizando lenguaje ensamblador. La solución propuesta permite seleccionar un cuadrante específico de una imagen en escala de grises y aplicar sobre él el algoritmo de interpolación, generando así una matriz de mayor resolución que conserva la coherencia visual de la imagen original mediante el cálculo de valores intermedios. El procesamiento de datos se realiza de manera exclusiva en ensamblador, garantizando un control total sobre las operaciones a nivel de arquitectura. La interacción con el usuario y la visualización de resultados se gestionan desde un entorno de alto nivel en Python, respetando la separación de responsabilidades establecida en la especificación del proyecto. Esta implementación permite profundizar en conceptos de arquitectura de computadores, manejo de memoria, optimización de código a bajo nivel y la integración con sistemas de alto nivel para aplicaciones de procesamiento de imágenes.

Palabras clave—x86, ARM, Interpolación Bilineal, Procesar Imágenes, Python, RISC-V

I. REQUERIMIENTOS DEL SISTEMA

- Sistema operativo: Linux (Ubuntu 20 recomendado)
- Simulador de arquitectura: QEMU (para ARM/x86) o entorno nativo.
- Python 3.6+ (librerías: OpenCV, Numpy, subprocess).
- Imágenes de entrada: formato de texto plano, escala de grises, tamaño 400x400 píxeles.
- Ensamblador nasm o compatible.
- Ejecución en arquitectura x86-64.

II. OPCIONES DE SOLUCIÓN DEL PROBLEMA

Se requiere desarrollar en ensamblador un programa que genera y trabaje con una imagen para procesarlas y aplicar interpolación bilineal a un sector de estas. Esto se plantea de la manera que el ensamblador recibe la imagen en formato crudo (un .txt), almacenando los píxeles en valores hexadecimal los cuales posteriormente son manipulados aritméticamente para conseguir los valores nuevos correspondientes a la imagen final.

Teniendo esto en cuenta, para el desarrollo del algoritmo de interpolación, se contó con las opciones de utilizar los simuladores ARM, RISC-V y x86, donde se debía seleccionar uno para trabajar en base a este. A continuación se presentan las consideraciones tomadas a la hora de seleccionar uno:

A. Implementación en ARM

Es una arquitectura es ampliamente utilizada en sistemas embebidos y dispositivos móviles, lo que permite simular y depurar de manera eficiente en entornos como QEMU. ARM dispone de un conjunto de instrucciones flexible y registros de propósito general suficientes, facilitando la manipulación de datos de imágenes de manera eficiente. Su rendimiento es consistente y cuenta con buena documentación y soporte en herramientas de desarrollo.

B. Implementación en x86-64

Esta arquitectura presenta ventajas al ejecutarse directamente en la mayoría de los PCs sin necesidad de emulación, y dispone de instrucciones especializadas para manejo de datos (por ejemplo, SSE, AVX). Sin embargo, trabajar con operaciones que requieren cálculos en punto flotante o interpolación en matrices puede implicar complejidad adicional en la administración de registros y optimización de acceso a memoria, especialmente si se busca mantener compatibilidad con entornos más limitados.

C. Implementación en RISC-V

Esta arquitectura tiene como fortaleza su simplicidad y extensibilidad, lo cual permite un control claro y predecible sobre las operaciones. RISC-V es ideal para proyectos educativos y de investigación, además de que ofrece un buen balance entre claridad de implementación y rendimiento. Sin embargo, su ecosistema es todavía joven en comparación con ARM y x86, por lo que las herramientas de simulación y optimización podrían ser menos maduras y afectar el tiempo de desarrollo.

Teniendo estas opciones, posteriormente se evaluó el trabajo que debía ser realizado por estos lenguajes, el cual se describe matemáticamente a continuación:

Teniendo la matriz original de tamaño 2x2:

$$\begin{bmatrix} w & x \\ y & z \end{bmatrix}$$

Se puede llegar al resultado de una matrix tamaño 4x4 de forma:

$$\begin{bmatrix} w & a & b & x \\ c & d & e & f \\ g & h & i & j \\ y & k & l & z \end{bmatrix}$$

Donde los valores generados tienen su origen en la aplicación de las siguientes operaciones:

$$a = \frac{2}{3} \times w + \frac{1}{3} \times x$$

$$b = \frac{1}{3} \times w + \frac{2}{3} \times x$$

$$c = \frac{2}{3} \times w + \frac{1}{3} \times y$$

$$d = \frac{2}{3} \times c + \frac{1}{3} \times f$$

$$e = \frac{1}{3} \times c + \frac{2}{3} \times f$$

$$f = \frac{2}{3} \times x + \frac{1}{3} \times z$$

$$g = \frac{1}{3} \times w + \frac{2}{3} \times y$$

$$h = \frac{1}{3} \times g + \frac{2}{3} \times j$$

$$i = \frac{1}{3} \times g + \frac{2}{3} \times j$$

$$j = \frac{1}{3} \times x + \frac{2}{3} \times z$$

$$k = \frac{2}{3} \times y + \frac{1}{3} \times z$$

III. COMPARACIÓN DE OPCIONES DE SOLUCION

Teniendo claro ya el objetivo y fin del programa, se puede realizar un análisis de que representa cada una de estas opciones, el cual se presenta a continuación resaltando cualidades de cada una de las posibilidades:

- La primera opción, que consiste en utilizar la arquitectura ARM, permite aprovechar un conjunto amplio de registros y modos de direccionamiento eficientes, lo que facilita la manipulación de las matrices de píxeles. Además, ARM permite un control preciso sobre los cálculos, utilizando representaciones en punto fijo para simplificar operaciones fraccionales como $1/3$ y $2/3$. La disponibilidad de herramientas de simulación como QEMU facilita el desarrollo y prueba del código sin requerir hardware físico.
- La segunda opción, que es la implementación en x86-64, permite ejecutar el código directamente sobre hardware de escritorio, simplificando las pruebas y depuración. Además, ofrece instrucciones especializadas para procesamiento de datos, que podrían acelerar los cálculos de interpolación. Sin embargo, su modelo de registros es más limitado y la gestión de datos en memoria puede ser menos eficiente al trabajar con matrices grandes.

- La tercera opción, que es la implementación en RISC-V cuenta con un conjunto de instrucciones es claro y predecible, lo que facilita el desarrollo de código estructurado y entendible. RISC-V permite trabajar de forma eficiente con operaciones básicas de suma y desplazamiento, adecuadas para la interpolación bilineal. Sin embargo, sus herramientas de desarrollo y simulación aún son menos maduras que en ARM o x86.

IV. SELECCION DE PROPUESTA FINAL

Una vez analizadas las opciones, se opta por trabajar con la arquitectura x86 para la implementación del algoritmo de interpolación bilineal se justifica ampliamente por su eficiencia y por las facilidades que ofrece para trabajar directamente con memoria, registros y punteros. Esta decisión permite que el procesamiento de datos se realice de manera clara y estructurada, facilitando tanto la depuración como la optimización.

El plan de implementación considera reservar espacio para una matriz de entrada de 100x100 píxeles y una matriz de salida de 200x200, representando la imagen interpolada. Se planea utilizar registros como rsi y rdi para manipular de forma eficiente las posiciones de memoria, lo cual es esencial para el manejo de imágenes.

Además, uno de los motivos principales de esta elección es el sistema flexible de punteros que proporciona x86 para recorrer y procesar las matrices. Los registros rsi y rdi actuarán como punteros a la memoria, facilitando la lectura y escritura de los datos de píxeles durante la interpolación. El cálculo de los índices, tanto para acceder a los elementos de la matriz original como para posicionar los resultados en la matriz de salida, se realizará mediante desplazamientos y multiplicaciones, optimizando el uso de la CPU.

También se contempla el uso de un flujo de entrada y salida de archivos eficiente. Se tiene en mente generar un buffer temporal para almacenar los datos leídos desde un archivo de entrada, los cuales luego se convertirán en una matriz de entrada mediante una rutina de conversión hexadecimal. Posteriormente, el resultado de la interpolación se almacenará en un archivo binario listo para ser visualizado.

Respecto al lenguaje de alto nivel, se ha decidido utilizar Python para desarrollar la interfaz con la que el usuario interactuará, ya que ofrece un entorno de trabajo simple y flexible, ideal para gestionar archivos e integrar distintas herramientas sin complicaciones; su compatibilidad con librerías como OpenCV (cv2) permite abrir, visualizar y manipular imágenes con muy pocas líneas de código, lo que simplifica el desarrollo y las pruebas. Además, cuenta con gran variedad de bibliotecas para generar interfaces gráficas, TKinter es una de las principales a considerar.